

Se dă un șir secret de N biți – $a_0, \dots, a_{N-1}$. La o operație, se alege o submulțime de poziții din șir și se inversează biții din șir de la pozițiile alese. Prin inversare un bit 0 se schimbă 1 și un bit 1 se schimbă în 0. După fiecare operație, ți se va da lungimea maximă a unei secvențe de biți de 1 din șirul modificat. **Operațiile sunt persistente, adică un bit inversat la o anumită operație, nu își schimbă starea până la o nouă operație care afectează acea poziție.**

Tu vrei să găsești unde este localizată cea mai lungă secvență de 1 după efectuarea tuturor operațiilor. Dacă există mai multe astfel de secvențe se poate găsi oricare dintre ele. Scrie un program care să realizeze scopul efectuând cât mai puține operații.

Detalii de implementare

Există T subteste pe test.

În programul tău trebuie să implementezi o funcție cu următorul prototip:

```
std::pair<int, int> find_longest_subarray_of_ones(int n);
```

Aceasta va fi apleată de T ori pe test și va primi prin parametrul N un număr întreg – lungimea șirului secret. Funcția va trebui să returneze perechea $pair\{l, r\}$, unde l este indicele celei mai din stânga poziții și r indicele celei mai din dreapta poziții pentru secvența determinată.

Trebuie să folosești funcția *flip_bits* pentru a face operațiile:

```
int flip_bits(const std::vector<bool> &flips);
```

Aceasta primește un vector de N biți $flips_0, \dots, flips_{n-1}$, unde $flips_i = 1$ înseamnă că a_i trebuie inversat. După inversarea biților solicitați, funcția returnează lungimea uneia dintre secvențele de lungime maximă conținând numai biți de 1 obținută după operație în șirul secret.

Trebuie să submitezi la sistemul de evaluare fișierul *ones.cpp* care trebuie să conțină funcția *find_longest_subarray_of_ones*. Acesta poate să conțină și alte linii de cod sau funcții ajutoare dar nu trebuie să conțină funcția *main*. Deasemenea, fișierul nu trebuie să conțină citiri de la intrarea standard sau afișări la ieșirea standard. Programul tău trebuie să include librăria *ones.h* prin instrucțiunea preprocesor:

```
#include "ones.h"
```

Testare locală

Îți este dat fișierul *Lgrader.cpp*, care poate fi compilat împreună cu programul tău pentru a-l testa. Acesta va citi numărul de subteste T în test, urmat de descrierea fiecăruia dintre acestea. Pentru fiecare subtest, mai întâi se va da un număr întreg N , după care, în linia următoare, vor urma $a_0 \dots a_{N-1}$. Dacă rulează corect, programul va returna 1 și numărul maximal de operații pe care le-ai solicitat. În caz contrar va afișa 0.

Restricții

$T = 5$

$1 \leq N \leq 10^4$

$$0 \leq a_i \leq 1$$

Punctare

Punctajul pe care îl vei obține pentru rezolvarea problemei depinde de numărul maximal de operații Q pe care programul tău le-a utilizat pentru a rezolva un singur subtest. Punctajul pe care îl vei obține pentru rezolvarea problemei este:

$$\left(\frac{85}{\max(85, Q)} \right)^{0.294} \times 100$$

Exemplu de interacțiune

Fie șirul initial este 1, 0, 1. Un model de interacțiune este:

Funcția participantului	Programul juriului
	apelează find_longest_subarray_of_ones(3)
apelează flip_bits({0, 0, 0})	Returnează valoarea 1
apelează flip_bits({0, 1, 0})	Returnează valoarea 3
Returnează valoarea {0, 2}	